WT-RHCA Networking

Complete Solutions Guide

November 23, 2025

Initial Network Configuration

Machine 1 (client.example.com)

```
# Set hostname
hostnamectl set-hostname client.example.com

# Configure network
nmcli con mod eth0 ipv4.addresses 192.168.1.50/24
nmcli con mod eth0 ipv4.gateway 192.168.1.1
nmcli con mod eth0 ipv4.dns 192.168.1.1
nmcli con mod eth0 ipv4.method manual
nmcli con up eth0

# Verify
ip addr show
ping 192.168.1.1
```

Machine 2 (server.example.com)

```
# Set hostname
hostnamectl set-hostname server.example.com

# Configure network
nmcli con mod eth0 ipv4.addresses 192.168.1.30/24
nmcli con mod eth0 ipv4.gateway 192.168.1.1
nmcli con mod eth0 ipv4.dns 192.168.1.1
nmcli con mod eth0 ipv4.method manual
nmcli con up eth0
```

1 Machine 1 Solutions

1. **Set multi-user target as default and reboot**

```
systemctl set-default multi-user.target
reboot
```

2. **Find lines containing ”seismic” and save to /root/wordlist**

```
grep seismic /usr/share/dict/words > /root/wordlist
```

3. **Create user David with password ageing**

```
# Create user
useradd David
echo "tekup" | passwd --stdin David

# Configure password ageing
chage -m 7 -M 28 -W 5 David

# Verify
chage -l David
```

4. **Create groups linuxadm and dba with GID 5000**

```
# Create linuxadm group
groupadd -g 5000 linuxadm

# Create dba group with same GID (use -o for non-unique)
groupadd -g 5000 -o dba

# Add David to linuxadm as secondary group
usermod -aG linuxadm David

# Verify
id David
getent group linuxadm
getent group dba
```

5. **Rename linuxadm to sysadm, change GID to 6000**

```
# Rename group and change GID
groupmod -n sysadm -g 6000 linuxadm

# Change David's primary group to sysadm
usermod -g sysadm David

# Verify
id David
getent group sysadm
```

6. **Create /archive directory with restricted permissions**

```
# Create directory with 770 permissions
mkdir /archive
chmod 770 /archive

# Create targets file with list of targets
systemctl list-unit-files --type=target > /archive/targets

# Verify
ls -ld /archive
ls -l /archive/targets
```

7. **Create user melinda with specific uid and gid**

```
# Create group with GID 3200 first
groupadd -g 3200 melgroup

# Create user with uid 4525 and gid 3200
useradd -u 4525 -g 3200 melinda
echo "redhat" | passwd --stdin melinda

# Verify
id melinda
```

8. **Set default permissions for melinda (files: 664, dirs: 764)**

```
# Files:  664 = umask 002
# Directories:  764 = umask 002 (but this gives 775 for dirs)
# For 764 dirs and 664 files, use umask 002

# Add to melinda's .bashrc
echo "umask 002" >> /home/melinda/.bashrc

# Note:  764 for directories requires manual chmod or ACLs
# umask 002 gives:  files 664, dirs 775
```

9. **Copy /archive to /root/archive2 with specific permissions**

```
# Copy recursively
cp -r /archive /root/archive2

# Set ownership
chown -R david:sysadm /root/archive2

# Set permissions:  david,sysadm=rwx, melinda=rx, others=x
chmod 2771 /root/archive2

# Set ACL for melinda (read/execute, no write/delete)
setfacl -R -m u:melinda:rx /root/archive2
setfacl -R -m d:u:melinda:rx /root/archive2

# Set sticky bit to prevent melinda from deleting
chmod +t /root/archive2

# Verify
ls -ld /root/archive2
getfacl /root/archive2
```

10. **Copy services list with specific permissions**

```
# Copy services list
systemctl list-units --type=service > /root/services

# Set permissions:  root=rwx(7), sysadm=x(1), melinda=r(4)+delete
# Base:  711
chmod 711 /root/services

# Set ACL for melinda (read + write for delete capability)
setfacl -m u:melinda:rw /root/services

# Verify
ls -l /root/services
getfacl /root/services
```

11. **Configure yum repositories**

```
# Create AppStream repo
cat > /etc/yum.repos.d/appstream.repo << EOF
[appstream]
name=AppStream
baseurl=http://tek-up.de/AppStream
enabled=1
gpgcheck=0
EOF

# Create BaseOS repo
cat > /etc/yum.repos.d/baseos.repo << EOF
[baseos]
name=BaseOS
baseurl=http://tek-up.de/BaseOS
enabled=1
gpgcheck=0
EOF

# Verify
dnf clean all
dnf repolist
```

12. **Create backup of /usr/local**

```
tar -cjf /root/backup.tar.bz2 /usr/local
```

13. **Configure autofs for /data from machine2**

```
# Install autofs
dnf install autofs nfs-utils -y

# Create mount point
mkdir -p /mnt/data

# Configure autofs
echo "/mnt /etc/auto.nfs" >> /etc/auto.master
echo "data -fstype=nfs,rw 192.168.1.30:/data" > /etc/auto.nfs

# Start and enable autofs
systemctl enable --now autofs

# Verify
ls /mnt/data
```

14. **Build pdf_conv image as user10**

```
# Create user10 if not exists
useradd user10

# Switch to user10
su - user10

# Install podman
sudo dnf install podman -y

# Create working directory
mkdir pdf_conv && cd pdf_conv

# Download files
curl -O https://raw.githubusercontent.com/sachinyadav3496/
Text-To-PDF/master/Dockerfile
curl -O https://raw.githubusercontent.com/sachinyadav3496/
Text-To-PDF/master/pdf_converter.py

# Build image
podman build -t pdf_conv .
```

15. **Create container with volumes**

```
# As user10, create host directories
sudo mkdir -p /action/incoming /action/outcoming
sudo chown user10:user10 /action/incoming /action/outcoming

# Run container
podman run -d --name convertor \
-v /action/incoming:/data/input:Z \
-v /action/outcoming:/data/output:Z \
pdf_conv
```

16. **Create systemd service for container**

```
# As user10, generate service file
podman generate systemd --name convertor --files

# Move to systemd user directory
mkdir -p ~/.config/systemd/user/
mv container-convertor.service ~/.config/systemd/user/
container-pdfconvertor.service

# Enable lingering for user10
sudo loginctl enable-linger user10

# Enable and start service
systemctl --user daemon-reload
systemctl --user enable container-pdfconvertor.service
systemctl --user start container-pdfconvertor.service
```

17. **Configure HTTP web server on port 87**

```
# Install httpd
dnf install httpd -y

# Configure port 87
sed -i 's/Listen 80/Listen 87/' /etc/httpd/conf/httpd.conf

# Configure SELinux
semanage port -a -t http_port_t -p tcp 87

# Configure firewall
firewall-cmd --permanent --add-port=87/tcp
firewall-cmd --reload

# Start and enable
systemctl enable --now httpd

# Verify
curl http://localhost:87
```

18. **Script to find files owned by user10**

```
#!/bin/bash
# Script to find files owned by user10
# Size:  greater than 10KB and less than 10M

find / -user user10 -type f -size +10k -size -10M \
-exec cp {} /tmp/find1/ \; 2>/dev/null

# Or to create directory and copy:
mkdir -p /tmp/find1
find / -user user10 -type f -size +10k -size -10M \
2>/dev/null > /tmp/find1/filelist.txt
```

19. **Configure NTP client**

```
# Install chrony
dnf install chrony -y

# Edit chrony.conf
echo "server server.example.com iburst" >> /etc/chrony.conf

# Or edit and replace:
sed -i '/^pool /d' /etc/chrony.conf
echo "server server.example.com iburst" >> /etc/chrony.conf

# Restart and enable
systemctl enable --now chronyd

# Verify
chronyc sources
chronyc tracking
```

2 Machine 2 Solutions

1. **Configure password complexity**

```
# Edit /etc/security/pwquality.conf
vi /etc/security/pwquality.conf

# Add/modify these lines:
minlen = 7
dcredit = -1
ucredit = -1
lcredit = -1

# Verify by creating test user
useradd testuser
passwd testuser
```

2. **Script to find files less than 2M from /usr**

```
#!/bin/bash
# /root/backup.sh

mkdir -p /root/backup
find /usr -type f -size -2M -exec cp --parents {} /root/backup/ \; 2>/dev/null

# Make executable
chmod +x /root/backup.sh
```

3. **Scheduled backup of /etc directory**

```
# Create backup script
cat > /root/backup.sh << 'EOF'
#!/bin/bash
DATE=$(date +%Y%m%d-%H%M%S)
tar -czf /root/etc-backup-$DATE.tar.gz /etc
EOF

# Make executable
chmod +x /root/backup.sh

# Schedule cron job (11 PM daily except Sunday)
crontab -e

# Add this line:
0 23 * * 1-6 /root/backup.sh

# Verify
crontab -l
```

4. **Create 2G swap partition**

```
# Create partition (assuming /dev/sdb1 available)
# Or use a file-based swap
dd if=/dev/zero of=/swapfile bs=1M count=2048
chmod 600 /swapfile

# Make swap
mkswap /swapfile

# Enable swap
swapon /swapfile

# Make permanent
echo "/swapfile none swap defaults 0 0" >> /etc/fstab

# Verify
swapon --show
free -h
```

5. **Create volume group and logical volumes**

```
# Create physical volume
pvcreate /dev/sdb

# Create volume group with 16MB PE size
vgcreate -s 16M vgfs /dev/sdb

# Create logical volumes
lvcreate -L 80M -n ext4vol vgfs
lvcreate -L 80M -n xfsvol vgfs

# Format with filesystems
mkfs.ext4 /dev/vgfs/ext4vol
mkfs.xfs /dev/vgfs/xfsvol

# Create mount points
mkdir /ext4vol /xfsvol

# Make permanent
echo "/dev/vgfs/ext4vol /ext4vol ext4 defaults 0 0" >> /etc/fstab
echo "/dev/vgfs/xfsvol /xfsvol xfs defaults 0 0" >> /etc/fstab

# Mount
mount -a

# Verify
df -h
lsblk
```

6. **Extend xfsvol to 157M-175M**

```
# Extend logical volume to 160M
lvextend -L 160M /dev/vgfs/xfsvol

# Resize XFS filesystem
xfs_growfs /xfsvol

# Verify
df -h /xfsvol
lvs
```

7. **Configure NFS server and autofs client**

```
# On Machine2 (server.example.com):
# Create user
useradd -u 3000 user30
passwd user30

# Install NFS
dnf install nfs-utils -y

# Configure NFS export
echo "/home 192.168.1.0/24(rw,sync,no_root_squash)" >> /etc/exports

# Start NFS
systemctl enable --now nfs-server
exportfs -av

# Configure firewall
firewall-cmd --permanent --add-service=nfs
firewall-cmd --reload

# On Machine1 (client.example.com):
# Create user without home
useradd -u 3000 -M -b /nfshome user30

# Install autofs
dnf install autofs nfs-utils -y

# Configure autofs
mkdir -p /nfshome
echo "/nfshome /etc/auto.home" >> /etc/auto.master
echo "user30 -fstype=nfs,rw 192.168.1.30:/home/user30" > /etc/auto.home

# Start autofs
systemctl enable --now autofs

# Test
su - user30
pwd
```

8. **Create VDO volume**

```
# Install VDO
dnf install vdo kmod-kvdo -y

# Create VDO volume
vdo create --name=vdo1 --device=/dev/sdc --vdoLogicalSize=16G

# Format with XFS
mkfs.xfs -K /dev/mapper/vdo1

# Create mount point
mkdir /xfsvdo1

# Make permanent
echo "/dev/mapper/vdo1 /xfsvdo1 xfs defaults,x-systemd.requires=vdo.service 0 0" >>
/etc/fstab

# Mount
mount -a

# Verify
df -h /xfsvdo1
vdostats --hu
```

9. **Set timezone and configure NTP**

```
# Set timezone (example:  America/New_York)
timedatectl set-timezone America/New_York

# Or list and choose:
timedatectl list-timezones

# Install and configure chrony
dnf install chrony -y

# Enable NTP
timedatectl set-ntp true

# Start chronyd
systemctl enable --now chronyd

# Verify
timedatectl
chronyc sources
```

10. **Create periodic job and restrict bob**

```
# Create cron job
crontab -e

# Add:  Every 5 minutes, Sun & Wed, 3-4am
/5 3 * * 0,3 date >> ~/tracking
/5 4 * * 0,3 date >> ~/tracking

# Or more precisely:
/5 3-4 * * 0,3 date >> ~/tracking

# Restrict bob from cron
echo "bob" >> /etc/cron.deny

# Verify
crontab -l
```

11. **Submit at job as user100**

```
# Create user100 if needed
useradd user100

# Switch to user100
su - user100

# Submit at job
at 23:30 03/31/2040 << EOF
date > /tmp/date.out 2>&1
EOF

# Or:
echo "date > /tmp/date.out 2>&1" | at 23:30 03/31/2040

# Verify
atq
at -c <job_number>
```

12. **Set recommended tuned profile**

```
# Install tuned
dnf install tuned -y

# Start and enable
systemctl enable --now tuned

# Get recommended profile
tuned-adm recommend

# Set recommended profile
tuned-adm profile $(tuned-adm recommend)

# Verify
tuned-adm active
```

Verification Checklist

- Reboot both machines and verify all mounts persist
- Test NFS/autofs by logging in as user30
- Verify all services start automatically
- Test firewall rules and SELinux contexts
- Verify cron jobs and at jobs are scheduled
- Check password complexity with test users
- Test container service after reboot